Mathematical Foundations of Stereo Vision Triangulation

2-Camera Distance Measurement, Quadratic Optimization, and Least Squares

Abstract

This document outlines the complete mathematical proof and methodology for building a classical stereo vision triangulation system. It covers the geometric formulation of skew lines, the derivation of the matrix-vector equation, the L_2 norm objective function, and the minimization of this error using both gradient descent and classical Ordinary Least Squares (OLS) regression.

1 The Geometry of Two Camera Rays

Imagine two cameras in 3D space. They both take a picture of the same object. We want to find the 3D coordinate of that object. Each camera shoots a "ray" from its sensor out through the lens into the real world.

- **Camera 1** is located at a 3D point $P_1 = [x_1, y_1, z_1]^T$. Its ray travels in a normalized direction vector $\vec{d_1} = [u_1, v_1, w_1]^T$.
- **Camera 2** is located at 3D point $P_2 = [x_2, y_2, z_2]^T$. Its ray travels in direction $\vec{d_2} = [u_2, v_2, w_2]^T$.

In 3D geometry, we define these lines using **parametric equations**. A point L on the line is defined by how far you travel along the direction vector (t_1 and t_2 are scalar multipliers representing the distance from the camera):

$$L_1(t_1) = P_1 + t_1\vec{d_1}$$
$$L_2(t_2) = P_2 + t_2\vec{d_2}$$

The Intersection Problem

If the lines intersect perfectly, there is some distance t_1 and t_2 where the points are exactly the same:

$$P_1 + t_1\vec{d_1} = P_2 + t_2\vec{d_2}$$

Let's rearrange this to put our unknowns (t_1, t_2) on the left, and our knowns (camera origins) on the right:

$$t_1\vec{d_1} - t_2\vec{d_2} = P_2 - P_1$$

2 Formulating the Matrix-Vector Equation ($Ax = b$)

We can write the equation above as a system of linear equations using matrices. Let's expand the vectors into their X, Y, and Z components:

$$t_1 \begin{bmatrix} u_1 \\ v_1 \\ w_1 \end{bmatrix} - t_2 \begin{bmatrix} u_2 \\ v_2 \\ w_2 \end{bmatrix} = \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \\ z_2 - z_1 \end{bmatrix}$$

Using the rules of matrix multiplication, we group the direction vectors into a 3×2 matrix, and the unknowns into a 2×1 vector:

$$\begin{bmatrix} u_1 & -u_2 \\ v_1 & -v_2 \\ w_1 & -w_2 \end{bmatrix} \begin{bmatrix} t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \\ z_2 - z_1 \end{bmatrix}$$

This is the classic linear algebra form: $Ax = b$

- A is our 3×2 matrix of ray directions.
- x is our 2×1 vector of unknowns $[t_1, t_2]^T$.
- b is our 3×1 vector of the difference between camera positions.

The Skew Line Problem: Notice that matrix A has 3 rows (3 equations) but only 2 columns (2 variables). This is an **overdetermined system**. Due to pixel noise, the rays will never perfectly intersect. Therefore, no exact solution for x exists. Ax will never exactly equal b.

3 The Error Function and L_2 Norm

Since Ax cannot equal b, we define a "residual" or "error" vector e:

$$e = Ax - b$$

We want to find the t_1 and t_2 that make this error as close to zero as possible by minimizing the length of the error vector. The standard way to measure vector length is the **Euclidean Norm (L_2 Norm)**:

$$\|e\|_2 = \sqrt{e_x^2 + e_y^2 + e_z^2}$$

Square roots make calculus difficult, so we choose to minimize the **Squared L_2 Norm** instead. Minimizing x^2 yields the exact same minimum as minimizing x.

The squared length of a vector e is the dot product of the vector with itself: $e \cdot e$. In matrix notation, this is the transpose of the vector multiplied by itself:

$$E(x) = \|e\|_2^2 = e^T e$$
$$E(x) = (Ax - b)^T (Ax - b)$$

This is our continuous loss function.

4 Quadratic Optimization (Expanding the Error)

Let's expand our error function $E(x)$ using the matrix algebra rule $(AB)^T = B^T A^T$:

$$E(x) = (x^T A^T - b^T)(Ax - b)$$
$$E(x) = x^T A^T Ax - x^T A^T b - b^T Ax + b^T b$$

Let's look at the dimensions of $x^T A^T b$. It evaluates to a 1×1 matrix (a scalar). Because the transpose of a scalar is just the same scalar, $(x^T A^T b)^T = b^T Ax$. Since $x^T A^T b$ and $b^T Ax$ are the exact same scalar number, we combine them:

$$\mathbf{E(x) = x^T A^T Ax - 2b^T Ax + b^T b}$$

This is a **Quadratic Form**. Because $E(x)$ expands into a multi-dimensional parabola (a bowl shape), it is convex, meaning it has one single global minimum.

5 Calculus & Manually Finding the Gradient

To find the bottom of the bowl using Gradient Descent, we need to know the slope (the derivative) of the bowl at any given point x. The multivariable derivative is called the **Gradient (∇)**.

$$\nabla E(x) = \frac{\partial}{\partial x}\left[x^T A^T Ax - 2b^T Ax + b^T b\right]$$

Applying matrix calculus rules:

1. The derivative of a constant ($b^T b$) is 0.
2. The derivative of a linear term $\frac{\partial}{\partial x}(-2b^T Ax) = -2A^T b$.
3. The derivative of a symmetric quadratic form $\frac{\partial}{\partial x}(x^T Qx) = 2Qx$. Here, $Q = A^T A$. So, the derivative is $2A^T Ax$.

Combining these, we get the exact gradient:

$$\mathbf{\nabla E(x) = 2A^T Ax - 2A^T b}$$

6 Manual Gradient Descent

Gradient descent works by picking a starting point, calculating the slope, and taking a small step in the opposite direction.

The Algorithm:

1. **Initialize:** Let your guess for $x = [0, 0]^T$ (Assume $t_1 = 0, t_2 = 0$).
2. **Define Learning Rate:** Choose a small scalar step size, $\alpha = 0.01$.
3. **Loop:**

$$\text{grad} = 2A^T Ax_{\text{old}} - 2A^T b$$
$$x_{\text{new}} = x_{\text{old}} - \alpha \cdot \text{grad}$$

4. **Stop:** Repeat the loop until $\|\text{grad}\|_2 < \epsilon$ (e.g., 0.0001).

Because the function is convex, this loop is mathematically guaranteed to converge.

7 The "Regression" Proof (Ordinary Least Squares)

In classical statistics, you don't use Gradient Descent to solve this because you can solve it analytically in one step. This is known as **Ordinary Least Squares (OLS) Regression**.

At the exact bottom of the bowl, the slope must be zero. Let's set our gradient to zero:

$$\begin{aligned} 2A^TAx - 2A^Tb &= 0 \\ A^TAx - A^Tb &= 0 \\ A^TAx &= A^Tb \end{aligned}$$

This last line is famously known as the **Normal Equations**. To isolate x, we multiply both sides by the inverse of (A^TA):

$$\mathbf{x} = (\mathbf{A^TA})^{-1}\mathbf{A^Tb}$$

Instead of an iterative loop, you can plug your matrices A and b into this equation, and it yields the exact optimal $[t_1, t_2]^T$ instantly.

8 Finding the Final 3D Point and Error Distance

Once you have computed the optimal vector $x = [t_1, t_2]^T$, you calculate the final geometry.

1. Find the closest points on the rays:

$$\begin{aligned} P_1^* &= P_1 + t_1\vec{d_1} \\ P_2^* &= P_2 + t_2\vec{d_2} \end{aligned}$$

2. Calculate the estimated 3D Point: Since the lines are skew, these points won't perfectly match. The best estimate for the actual 3D object is the exact midpoint between them:

$$\text{Estimated 3D Point} = \frac{P_1^* + P_2^*}{2}$$

3. Calculate the Measurement Error: How far apart did the rays end up? This is the L_2 norm distance between the two closest points:

$$\text{Error Distance} = \sqrt{(P_{1x}^* - P_{2x}^*)^2 + (P_{1y}^* - P_{2y}^*)^2 + (P_{1z}^* - P_{2z}^*)^2}$$

This pipeline mathematically guarantees the optimal triangulation of a 3D point from two 2D camera views.

0.1 Pixel and Normalized Coordinates

In standard computer vision, a pixel is represented by its coordinates (u, v). However, working directly in pixels is numerically unstable for optimization. We transform these into *normalized coordinates* (u_n, v_n) relative to the optical center (C_X, C_Y):

$$u_n = \frac{u - C_X}{C_X}, \quad v_n = \frac{v - C_Y}{C_Y} \tag{1}$$

This mapping ensures that the coordinates are centered at 0 and generally fall within the range $[-1, 1]$. Normalization is crucial because it decouples the image resolution from the mathematical optimization space, preventing gradient explosion.

0.2 The Ray Equation

A camera maps a 3D point to a 2D pixel. Inversely, a pixel (u, v) defines a 3D ray shooting out from the camera's optical center. In our model, the ray vector $\mathbf{r}$ is defined as:

$$\mathbf{r} = \begin{bmatrix} u_n \gamma_x \\ v_n \gamma_y \\ 1 \end{bmatrix} \tag{2}$$

Here, γ_x and γ_y act as the effective inverse focal lengths (scaled by C_X, C_Y). In a perfect pinhole camera, $\gamma_x = C_X / f_x$. By making γ variable across the image, we inherently model lens distortion without complex polynomials.

1 Bilinear Interpolation Mathematics

To model non-linear lens distortion smoothly, the image is divided into a grid (e.g., 3×3). Trainable parameters g_{ij} are placed at the nodes of this grid. For any arbitrary pixel (u, v), the local gamma value is computed using bilinear interpolation.

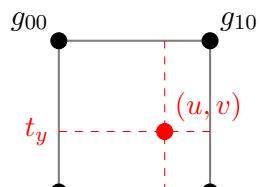
First, we map (u, v) to continuous grid coordinates (f_x, f_y):

$$f_x = \frac{u}{\text{CELL_W}}, \quad f_y = \frac{v}{\text{CELL_H}} \tag{3}$$

We find the top-left integer node (i_x, i_y) and the fractional distances $t_x = f_x - i_x$ and $t_y = f_y - i_y$. The interpolated value $g(u, v)$ is a weighted sum of the four surrounding nodes:

$$g(u, v) = \underbrace{(1 - t_x)(1 - t_y)}_{w_{00}} g_{00} + \underbrace{t_x(1 - t_y)}_{w_{10}} g_{10} + \underbrace{(1 - t_x)t_y}_{w_{01}} g_{01} + \underbrace{t_x t_y}_{w_{11}} g_{11} \tag{4}$$

This approach guarantees C^0 continuity across the image, meaning there are no harsh jumps in distortion correction, while remaining computationally extremely cheap (Big-O $\mathcal{O}(1)$ per pixel).



3. **Decoupled Gradients:** The x-axis parameters strictly handle horizontal disparity (depth), while the y-axis parameters strictly handle epipolar alignment (camera rotation/vertical shift).

Thus, at each grid node, the parameter tensor Γ has shape $(R, C, 2)$, storing both γ_x and γ_y.

3 Stereo Geometry and Triangulation

Given a matched point in the left (u_L, v_L) and right (u_R, v_R) images, we generate two 3D rays: $\mathbf{v}_L$ and $\mathbf{v}_R$. Let Z be the depth along the left camera's optical axis, and Z_R be the depth along the right camera's axis.

The 3D point in the left camera frame is $P_L = Z\mathbf{v}_L$. The same point in the right camera frame is $P_R = Z_R\mathbf{v}_R$. The cameras are separated by a baseline vector $\mathbf{d} = [d_x, d_y, 0]^T$. The geometric relationship is:

$$P_L = \mathbf{d} + P_R \implies Z\mathbf{v}_L - Z_R\mathbf{v}_R = \mathbf{d} \tag{5}$$

In matrix form:

$$\underbrace{\begin{bmatrix} | & | \\ \mathbf{v}_L & -\mathbf{v}_R \\ | & | \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} Z \\ Z_R \end{bmatrix}}_{\mathbf{x}} = \mathbf{d} \tag{6}$$

Because rays in the real world rarely intersect perfectly due to noise, this system is overdetermined (3 equations, 2 unknowns). We solve it using the Ordinary Least Squares (OLS) method:

$$\mathbf{A}^T\mathbf{A}\mathbf{x} = \mathbf{A}^T\mathbf{d} \implies \mathbf{x} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{d} \tag{7}$$

The estimated depth is simply $Z = x_1$.

4 Disparity Mathematics

Disparity is the difference in image coordinates of the same 3D point. In our ray space, the horizontal disparity is:

$$\text{disp} = v_{Lx} - v_{Rx} \tag{8}$$

From the triangulation equation, if $Z \approx Z_R$ (which is true when $Z \gg d_x$), we have $Z(v_{Lx} - v_{Rx}) \approx d_x$. Thus:

$$\text{disp} \approx \frac{d_x}{Z} \tag{9}$$

This fundamental inverse relationship explains why stereo vision is highly accurate for near objects (small $Z \implies$ large disparity) but loses precision for far objects (large $Z \implies$ disparity approaches zero).

5 Loss Function Deep Derivation

To train our Γ grid and baseline d, we define a custom loss function $\mathcal{L} = \mathcal{L}_{1D} + \mathcal{L}_Y$.

5.1 Why avoid Z-space loss?

A naive approach would be $\mathcal{L} = (Z_{pred} - Z_{true})^2$. However, since $Z \propto 1/\text{disp}$, a small pixel error at a large distance causes a massive Z error (Z^2 blow-up). This makes gradients highly unstable.

5.2 Relative Disparity Loss ($\mathcal{L}_{1D}$)

Instead, we optimize in ray-disparity space. The true expected ray disparity is $\text{disp}_{true} = d_x / Z_{true}$. We define the relative error as:

$$e_x = \frac{v_{Lx} - v_{Rx}}{\text{disp}_{true}} - 1 = \frac{(v_{Lx} - v_{Rx}) Z_{true}}{d_x} - 1 \tag{10}$$

The loss is:

$$\mathcal{L}_{1D} = \frac{1}{2} e_x^2 \tag{11}$$

This relative formulation treats a 5% depth error at 1 meter the same as a 5% depth error at 10 meters, stabilizing the optimization.

5.3 Epipolar Y-Alignment Loss ($\mathcal{L}_Y$)

For perfect stereo, the vertical ray components should perfectly match the physical vertical baseline d_y.

$$e_y = (v_{Ly} - v_{Ry}) - \frac{d_y}{Z_{true}} \tag{12}$$

$$\mathcal{L}_Y = \frac{1}{2} e_y^2 \tag{13}$$

This forces the network to learn γ_y such that epipolar lines become perfectly horizontal in the ray space.

6 Huber Loss Robustness

Human clicks during calibration contain outliers. Standard Mean Squared Error (MSE) heavily penalizes large errors, pulling the model toward bad data points. We wrap our errors $e \in \{e_x, e_y\}$ in a Huber loss function $\mathcal{L}_\delta(e)$:

$$\mathcal{L}_\delta(e) = \begin{cases} \frac{1}{2} e^2 & \text{if } |e| \leq \delta \\ \delta \left(|e| - \frac{1}{2}\delta \right) & \text{if } |e| > \delta \end{cases} \tag{14}$$

With derivative:

$$\frac{\partial \mathcal{L}_\delta}{\partial e} = \begin{cases} e & \text{if } |e| \leq \delta \\ \delta \cdot \text{sign}(e) & \text{if } |e| > \delta \end{cases} \tag{15}$$

In the code, $\delta = 0.05$ (a 5% relative error threshold). Beyond this, the gradient magnitude is capped at δ, making the model highly robust to misclicks.

7 Gradient Derivations

Because we do not use automatic differentiation (like PyTorch), we must derive all gradients analytically using the chain rule.

7.1 Gradients with respect to Ray Vectors

Let $D = v_{Lx} - v_{Rx}$. Recall $e_x = \frac{DZ_{true}}{d_x} - 1$. Assuming $|e_x| \leq \delta$:

$$\frac{\partial \mathcal{L}}{\partial D} = e_x \frac{Z_{true}}{d_x} \tag{16}$$

Since $D = v_{Lx} - v_{Rx}$, we have:

$$\frac{\partial \mathcal{L}}{\partial v_{Lx}} = \frac{\partial \mathcal{L}}{\partial D}, \quad \frac{\partial \mathcal{L}}{\partial v_{Rx}} = -\frac{\partial \mathcal{L}}{\partial D} \tag{17}$$

Similarly for Y:

$$\frac{\partial \mathcal{L}}{\partial v_{Ly}} = e_y, \quad \frac{\partial \mathcal{L}}{\partial v_{Ry}} = -e_y \tag{18}$$

7.2 Gradients with respect to Grid Nodes (Γ)

By the ray equation $v_{Lx} = u_{nL}\gamma_{xL}$. Thus:

$$\frac{\partial v_{Lx}}{\partial \gamma_{xL}} = u_{nL} \tag{19}$$

Using bilinear interpolation, $\gamma_{xL} = \sum_{k=1}^{4} w_k g_k$. Therefore, the gradient flows to the four surrounding grid nodes g_k:

$$\frac{\partial \mathcal{L}}{\partial g_k} = \frac{\partial \mathcal{L}}{\partial v_{Lx}} \frac{\partial v_{Lx}}{\partial \gamma_{xL}} \frac{\partial \gamma_{xL}}{\partial g_k} = \left(\frac{\partial \mathcal{L}}{\partial v_{Lx}} \right) \cdot u_{nL} \cdot w_k \tag{20}$$

This exact logic is implemented in the `forward_grads` Python function.

7.3 Gradients with respect to Baseline

$$\frac{\partial \mathcal{L}}{\partial d_x} = e_x \frac{\partial e_x}{\partial d_x} = e_x \left(-\frac{DZ_{true}}{d_x^2} \right) \tag{21}$$

$$\frac{\partial \mathcal{L}}{\partial d_y} = e_y \left(-\frac{1}{Z_{true}} \right) \tag{22}$$